

EDUMIND AI

Smart AI-Powered Learning Assistant

PROJECT DOCUMENTATION

Academic / Project Submission Document

Prepared for project demonstration and submission

Date: 29 September 2026

Document Control

Item	Details
Project Name	EduMind AI
Project Type	AI-powered educational learning assistant
Primary Backend	Python + FastAPI
AI Service	Google Gemini API / Google Gen AI Python SDK
Frontend	HTML / CSS / JavaScript interface
Current Project Folder	C:\Users\acer\Desktop\EDI GENIE_AI
Primary Use Case	Interactive question answering and learning support
Documentation Purpose	Project overview, architecture, setup, modules, API usage, and operation

Table of Contents

- 1. Abstract
- 2. Introduction
- 3. Problem Statement
- 4. Objectives
- 5. Scope of the Project
- 6. Key Features
- 7. System Architecture
- 8. Technology Stack
- 9. Project Folder Structure
- 10. Module Description
- 11. Application Workflow
- 12. API Endpoints
- 13. Installation and Setup
- 14. Running the Application
- 15. Testing and Validation
- 16. Security and API Key Management
- 17. Current Limitations
- 18. Future Enhancements
- 19. Conclusion
- 20. Sample Demonstration Questions

1. Abstract

EduMind AI is a smart, AI-powered educational learning assistant designed to support students through interactive learning features. The application combines question answering, concept explanation, quiz generation, study-note generation, text summarization, and learning-path recommendations in a single user interface.

The application uses a Python and FastAPI backend to receive requests from the frontend and route them to modular services. Google Gemini is used as the generative AI service. The design separates the user interface, backend routes, learning modules, and AI client so that each part can be maintained independently.

The project is intended to demonstrate a practical use of Generative AI in education and to provide a simple interface that can be used by students and beginners.

2. Introduction

Students often use separate tools for asking questions, generating study notes, revising topics, and practicing with quizzes. EduMind AI brings these functions into one educational assistant so that learning activities can be accessed through a common interface.

The current implementation is organized around a FastAPI web application. The frontend submits user requests to backend endpoints. The backend then calls the appropriate learning module, which can create a structured prompt and use the Gemini API to generate the response.

3. Problem Statement

Students may face difficulty understanding technical concepts, preparing notes quickly, revising large amounts of text, and identifying an appropriate learning sequence. A single educational assistant can reduce the need to switch between multiple tools and can provide interactive support on demand.

The project addresses this problem by providing a unified AI-based learning interface with modular educational functions.

4. Objectives

- Provide interactive AI-powered question answering for general learning and programming questions.
- Explain complex concepts in simple, beginner-friendly language.
- Generate quizzes for practice and revision.
- Create structured study notes from a topic.

- Summarize longer educational content into concise information.
- Recommend a structured learning path based on topic and learner level.
- Demonstrate how Generative AI can be integrated into a web-based educational system.
- Keep AI credentials outside the application source code by using environment variables.

5. Scope of the Project

The project focuses on an interactive educational assistant. Its current scope includes the frontend interface, FastAPI backend, modular learning functions, Gemini API integration, and local development and testing through Uvicorn.

The project does not currently attempt to provide a complete learning management system, formal student assessment records, institutional authentication, or a production-grade multi-user database.

6. Key Features

6.1 Question Answering

Users can enter questions such as “What is Python?” or other learning and programming questions. The Q&A module builds an instructional prompt and sends it to the Gemini service.

6.2 Concept Explanation

The explanation feature takes a topic or concept and asks the AI service to provide a simple explanation suitable for a learner or beginner.

6.3 Quiz Generation

The quiz feature generates practice questions from a supplied topic. It can be used for self-assessment and revision.

6.4 Study Notes

The notes module produces structured notes that can include an introduction, key points, important terms, examples, and quick revision content.

6.5 Summarization

The summarization feature accepts educational text and produces a shorter version that retains important information.

6.6 Learning Path Recommendation

The learning-path feature accepts a topic and learner level, then requests a structured learning sequence from the AI service.

7. System Architecture

EduMind AI follows a simple layered architecture:

- **Presentation Layer** – HTML/CSS/JavaScript interface used by the student.
- **Application Layer** – FastAPI routes in main.py receive HTTP requests and return responses.
- **Learning Module Layer** – qna.py, explanation_module.py, quiz_module.py, notes_module.py, summary_module.py, and learning_path.py prepare feature-specific prompts and call the AI client.
- **AI Integration Layer** – gemini_client.py loads the environment variable, creates the Google Gen AI client, and sends generate-content requests to the configured Gemini model.
- **External AI Service** – Google Gemini API processes prompts and returns generated text.

The overall request flow is: User → Frontend → FastAPI Endpoint → Feature Module → Gemini Client → Gemini API → Response → Frontend.

8. Technology Stack

Technology	Purpose
Python 3.10	Backend application and AI integration
FastAPI	Web framework for REST-style endpoints
Uvicorn	ASGI server used to run the FastAPI application locally
Google Gen AI Python SDK	Programmatic access to Gemini models
Google Gemini API	Generative AI response generation
HTML / CSS / JavaScript	Frontend user interface and browser interaction
python-dotenv	Loading environment variables from .env
VS Code	Development environment

9. Project Folder Structure

The current project is organized as follows:

```

EDI GENIE_AI/
|
├── .env
├── explanation_module.py
├── gemini_client.py
├── learning_path.py
├── main.py
├── notes_module.py
└── qna.py

```

```

├── quiz_module.py
├── summary_module.py
├── requirements.txt
├──
├── static/
├──
├── templates/
│   ├── index.html
│   └── index_backup.html

```

The .env file stores the Gemini API credential and should not be uploaded with a real secret. The templates folder contains the main HTML interface. The static folder is used for frontend assets. The Python modules implement backend routes and learning logic.

10. Module Description

File / Module	Responsibility
main.py	Creates the FastAPI application, mounts static files, loads the HTML template, and defines application endpoints.
gemini_client.py	Loads the Gemini API key, creates the AI client, selects the configured model, and sends generate-content requests.
qna.py	Creates the prompt used for general educational question answering.
explanation_module.py	Creates prompts for simple concept explanations.
quiz_module.py	Creates prompts for quiz generation.
notes_module.py	Creates structured study-note prompts.
summary_module.py	Creates prompts for content summarization.
learning_path.py	Creates prompts for topic and learner-level based learning recommendations.
templates/index.html	Main browser interface for interacting with the application.
static/	Frontend assets such as CSS or JavaScript files, when present.
requirements.txt	Python dependencies required by the project.

11. Application Workflow

11.1 General Q&A Workflow

1. The user opens the EduMind AI web page in a browser.
2. The user enters a question and submits it.
3. The frontend sends the question to the /qa endpoint.
4. main.py calls answer_question() from qna.py.
5. qna.py creates an instructional prompt.
6. generate_content() in gemini_client.py sends the prompt to the Gemini model.
7. The AI response is returned through the backend.
8. The frontend displays the answer to the user.

11.2 Other Learning Workflows

The same general pattern is used for explanation, quiz, notes, summary, and learning-path operations. Each feature has its own endpoint and module so that changes to one feature do not require rewriting the entire application.

12. API Endpoints

Method	Endpoint	Purpose
GET	/	Loads the main EduMind AI interface.
GET	/health	Returns basic application health information.
POST	/qa	Answers a user question.
POST	/explain	Explains a supplied concept or text.
POST	/quiz	Generates a quiz from a topic or text.
POST	/notes	Generates study notes.
POST	/summarize	Summarizes supplied text.
POST	/learn/recommendations	Generates a learning path using topic and level.

13. Installation and Setup

13.1 Prerequisites

- Windows computer with Python installed.
- Visual Studio Code or another Python-capable editor.

- Internet access for Gemini API calls.
- A valid Gemini API key and a Google AI Studio project with API access.

13.2 Install Dependencies

From the project folder, install the dependencies listed in requirements.txt. A typical local setup uses:

```
python -m pip install -r requirements.txt
```

13.3 Configure .env

Create or edit the .env file in the project root. Store the API key as an environment variable. Do not hard-code the real key into Python source files and do not publish the key in screenshots, repositories, or documentation.

```
GEMINI_API_KEY=YOUR_ACTUAL_API_KEY
```

13.4 Start the Server

```
python -m uvicorn main:app --reload
```

14. Running the Application

9. Open the project folder in VS Code.
10. Open the VS Code terminal.
11. Activate the appropriate Python environment if one is being used.
12. Run `python -m uvicorn main:app --reload`.
13. Wait for the message indicating that Uvicorn is running on `http://127.0.0.1:8000`.
14. Open `http://127.0.0.1:8000` in a web browser.
15. Use the question-answer and other learning features from the interface.

For a presentation demonstration, a simple question such as “What is Python?” can be used to show the end-to-end request and response flow.

15. Testing and Validation

15.1 Basic Functional Testing

Test	Expected Result
Open /	Main EduMind AI page loads.
GET /health	Health response is returned.
POST /qa with a valid question	AI answer is returned when Gemini access is available.
POST /explain	Explanation text is returned.
POST /quiz	Quiz content is returned.
POST /notes	Structured notes are returned.

POST /summarize	Summary text is returned.
POST /learn/recommendations	Learning recommendations are returned.

16. Security and API Key Management

The Gemini API key is a sensitive credential. The project is designed to load the key from the `.env` file rather than embedding it directly in source code.

- Never publish the real API key in a Git repository.
- Never paste the key into a project report, video, or screenshot.
- Use `.gitignore` to exclude `.env` from source-control uploads.
- If a key is exposed publicly, revoke or replace it through the relevant Google API-key management interface.
- Keep demonstration material free of secrets and personal credentials.

The application should also validate empty prompts and handle AI-service errors so that a user receives a meaningful error instead of an unhandled crash.

17. Current Limitations

- The application depends on external Gemini API access and service availability.
- Free-tier quotas, rate limits, model availability, or project access restrictions can affect AI requests.
- The current project is primarily a local development application and is not yet a full production deployment.
- There is no persistent student account or learning-history database in the current structure.
- Generated answers depend on the quality and availability of the underlying AI model.

During development, Gemini API access can also produce authentication, permission, quota, or temporary availability errors. These are external service conditions and should be distinguished from local FastAPI or Python errors.

18. Future Enhancements

- Add student login and personalized learning profiles.
- Store learning history, quiz results, and revision progress in a database.
- Add multilingual responses including Tamil and English.
- Add document upload for PDF or classroom-material analysis.
- Add speech input and text-to-speech output.
- Add teacher dashboards and assignment support.

- Add better error handling, logging, monitoring, and production deployment.
- Add unit tests and automated API tests for each learning module.

19. Conclusion

EduMind AI demonstrates how Generative AI can be integrated into an educational web application. By combining question answering, explanations, quizzes, notes, summaries, and learning-path recommendations, the project provides a single interface for multiple learning activities.

The modular backend structure makes the project easier to understand, test, and extend. The use of environment variables for API credentials also supports safer development practices. With additional authentication, persistent data, testing, and production deployment, EduMind AI can be extended into a more complete educational platform.

20. Sample Demonstration Questions

Feature	Example Input
Question Answering	What is Python?
Explanation	Explain object-oriented programming in simple terms.
Quiz	Generate a beginner quiz on Python loops.
Study Notes	Create study notes for database management systems.
Summarization	Summarize the following chapter text.
Learning Path	Give me a beginner learning path for Python.

End of Document